



1

マスキングテープ (Masking Tape)

Author : 萩原千晴

どの小課題についても、問題文に書いてある処理の流れを順に追っていけば良い。

小課題 1

$q_k = 1$ ($1 \leq k \leq N$) であるため、マスキングテープを貼ることはなく、常に色の更新が行われる。 $H = 2, W = 2$ であるため、マスキングテープを貼る度に 2×2 の 4 マスすべてに新たな色が塗られる。 Q 回の操作を順に行っていくことを考えると、最終的な 4 マスの色は最後の N 回目の操作で更新した色になっていることがわかる。 よって、 c_Q を 2 つずつ空白区切りで、2 行に出力すれば得点を得られる。

小課題 2

この小課題では、紙の大きさについては与えられた制約通りであり、マスキングテープを貼ることなく常に色の更新を行う。 $H \times W$ の二次元配列を用意し、操作が行われる度に 4 つのマスの色を更新する。最後に、この二次元配列を出力すれば得点が得られる。

なお、今回 $(x_k, y_k), (x_k, y_k + 1), (x_k + 1, y_k), (x_k + 1, y_k + 1)$ を更新しており、順にこのように書いても良いが、配列 dx, dy を $dx = 0, 1, 0, 1, dy = 0, 0, 1, 1$ のように定義することで、4 重ループを回せば $(x_k + dx_i, y_k + dy_i)$ ($1 \leq i \leq 4$) のように表すことができる。グリッド上で移動の操作などをするときには便利な方法なので、知らなかった方は覚えておくと今後使えるかもしれない。

小課題 3

マスキングテープを貼る、という処理を考える必要が出てくる。この処理をどう扱うかということ、新たな $H \times W$ で値が 0 または 1 の二次元配列 `masked` を用意し、これにマスキングテープが貼られたかどうかの情報を保存する。値が 0 ならばマスキングテープが貼られていない、1 ならば貼られていることを表すとする。

$q_k = 1$, すなわち色を更新したいときは、更新しようとするマスに対応する `masked` の値を確認して、0 ならばマスキングテープが貼られていないので更新し、1 ならばマスキングテープが貼られているので更新しない。

$q_k = 2$, すなわちマスキングテープを貼るときは、`masked` の対応する値を更新する。最後に、処理を行っていた二次元配列を出力すれば得点が得られる。



2

ビリヤード (Billiards)

Author : 井上誠大

小課題 1

この小課題では、全ての i について $P_i = -1$ である。よって、ビ太郎は $A_i \leq X$ なるボール i のうち i が最大となるボールをひとつだけ落とせばよい。

小課題 2

この小課題では、ボールを必ずボール 1, ボール 2, ..., ボール N の順で落とす必要がある。このとき、ボール i を落とすために必要な集中力の合計は $A_1 + A_2 + \dots + A_i$ である。よって、 $A_1 + A_2 + \dots + A_i \leq X$ を満たす最大の i について、ボール 1, ボール 2, ..., ボール i を落とせばよい。

小課題 3

全てのボール i について、ボール i を落とすために落とす必要のあるボールを遡っていき（ボール i を落とすためにはボール $j = P_i$ を落とす必要があり、ボール j を落とすためにはボール $k = P_j$ を落とす必要があり、... という要領）、それらのボールを落とすのに必要な集中力の合計が X 以下ならばボール i を落とすことができるという方法で判定を行えばよい。 $P_i < i$ なので、遡る過程において N ステップ以内に $P_i = -1$ なるボールに到達して停止する。時間計算量は $O(N^2)$ である。

小課題 4

i が小さい方から順に、ボール i を落とすために必要な集中力 $dp[i]$ を動的計画法により求める。この $dp[i]$ は次の通りに求めることができる。

- $dp[i] = A_i$ ($P_i = -1$ である場合)
- $dp[i] = A_i + dp[P_i]$ ($P_i \neq -1$ である場合)

あとは、 $dp[i] \leq X$ なる最大の i を求めればよい。時間計算量は $O(N)$ である。



第 24 回日本情報オリンピック (JOI 2024/2025) 二次予選
2024 年 12 月 8 日 (オンライン開催)

小課題 5

小課題 3 同様に、ボール i を落とすために落とす必要のあるボールを遡っていく。しかし、遡っていく過程で、ボール k を落とすためにボール k を落とすことが要求されることがある（この k は i と同じであるとは限らないことに注意せよ）。このような循環依存が出現した場合、ボール i を落とすことはできない。

この判定は、今までに落とすことを要求されたボールの集合 S を保持し、ボール i からボール P_i へと遡っていく際に P_i が S に含まれるかどうかを判定すればよい。この判定はバケットソートの要領で $O(1)$ で実現可能である。

この判定を小課題 3 のプログラムに書き加えることで、時間計算量 $O(N^2)$ でこの小課題に正解することができる。

満点

小課題 5 同様に、ボール i を落とすために落とす必要のあるボールを遡っていきながら循環依存を検知し、各ボール i について、「ボール i を落とすことができるか」「落とすために必要な集中力 $dp[i]$ 」の 2 つを動的計画法により求める。これは、以下のようなアルゴリズムで実現可能である。

- $P_i = -1$ であるなら、ボール i を落とすことができ、 $dp[i] = A_i$ である。
- そうでなく P_i へと遡る際に循環依存が検出されたなら、ボール i を落とすことはできない。
- そうでなく P_i が落とすことのできるボールであると判明しているなら、ボール i を落とすことができ、 $dp[i] = A_i + dp[P_i]$ である。
- そうでなく P_i が落とすことのできないボールであると判明しているなら、ボール i を落とすことはできない。
- そうでないなら、ボール P_i について先に求解し、その結果を利用してボール i に関して「ボール i を落とすことができるか」と「 $dp[i]$ 」とを求める。

このような手続きを踏むことで、ボール i からボール P_i に遡る行為はアルゴリズム全体で高々 1 度しか行われないう。よって、時間計算量 $O(N)$ でこの問題に正解できる。



3

ソフトクリーム (Softcream)

Author : 平田誠治 (define)

小課題 1

この小課題では $X = Y = 1$ より選択の余地は Alice のトッピングのみである。したがって、すべてのトッピングを試して、最もスコアの高くなるものを選択すれば良い。時間計算量は $O(Z)$ である。

```
1      int answer=0;
2      for(int c:C){
3          int price=A[0]+B[0]+c;
4          if(abs(price-P)>answer){
5              answer=abs(price-P);
6          }
7      }
```

小課題 2

この小課題ではトッピングに加えて、コーンにも選択の余地がある。両者が最善を尽くすという設定であるから、Bob はコーンの選択の時点で Alice がスコアを最大化するトッピングを選択する事を考慮しなければならない。

$g(b)$ を Bob が値段 b のコーンを選択した仮定のもと、Alice のトッピングの選択によって取りうるスコアの最大値とする。Bob は $g(b)$ が最小となるようなコーンを選択すれば良い。 $g(b)$ の計算は、フレーバーとコーンを固定した場合を考えているので小課題 1 に帰着される。

j ($1 \leq j \leq Y$) について $g(b)$ の計算に $O(Z)$ かかるから、全体の時間計算量は $O(YZ)$ である。



第 24 回日本情報オリンピック (JOI 2024/2025) 二次予選
2024 年 12 月 8 日 (オンライン開催)

```
1      int answer=(int)(3e8);
2      for(int b:B){
3          int g=0;
4          for(int c:C){
5              int price=A[0]+b+c;
6              if(abs(price-P)>g){
7                  g=abs(price-P);
8              }
9          }
10         if(answer>g){
11             answer=g;
12         }
13     }
```

小課題 3

この小課題ではフレーバーにも選択の余地がある。小課題 2 と同様にして、Alice はフレーバーの選択の時点で Bob がスコアを最小化するコーンを選択する事を考慮しなければならない。

$f(a)$ を Alice が値段 a のフレーバーを選択した仮定のもと、Bob のコーンの選択によって取りうるスコアの最小値とする。Alice は $f(a)$ が最大となるようなフレーバーを選択すれば良い。 $f(a)$ の計算は、フレーバーを固定した場合を考えているので小課題 2 に帰着される。

i ($1 \leq i \leq X$) について $f(a)$ の計算に $O(YZ)$ かかるから、全体の時間計算量は $O(XYZ)$ である。このアルゴリズムは Mini-Max 法として知られるものである。



第 24 回日本情報オリンピック (JOI 2024/2025) 二次予選
2024 年 12 月 8 日 (オンライン開催)

```
1      int answer=0;
2      for(int a:A){
3          int f=(int)(3e8);
4          for(int b:B){
5              int g=0;
6              for(int c:C){
7                  int price=a+b+c;
8                  if(abs(price-P)>g){
9                      g=abs(price-P);
10                 }
11             }
12             if(f>g){
13                 f=g;
14             }
15         }
16         if(f>answer){
17             answer=f;
18         }
19     }
```

小課題 4

小課題 3 のアルゴリズムを高速化する事を考える.

実は Alice のトッピングの選択は, 値段が最小/最大のもののみを考慮すれば十分である. Alice の目標は値段の合計と P との絶対値であるスコアを最大化する事であるから, 値段が最小のものと最大のもののうち, スコアがより高い方を選択すれば良い.

全体の時間計算量は $O(XY)$ である.



第 24 回日本情報オリンピック (JOI 2024/2025) 二次予選
2024 年 12 月 8 日 (オンライン開催)

満点

小課題 4 ではトッピングの選択にかかる計算量を削減したが、ここではコーンの選択を高速化する。
フレーバーを値段 a に固定した際に、値段 b のコーンを選択した際の最終的なスコアは

$$\max(|a + b + C_{\min} - P|, |a + b + C_{\max} - P|) = \max(|b - (P - a - C_{\min})|, |b - (P - a - C_{\max})|)$$

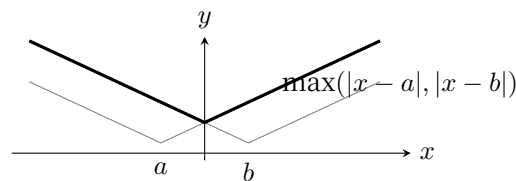
である。一般に

$$\max(|x - a|, |x - b|) = \frac{|b - a|}{2} + \left| x - \frac{a + b}{2} \right|$$

であるから、

$$\max(|b - (P - a - C_{\min})|, |b - (P - a - C_{\max})|) = \frac{C_{\max} - C_{\min}}{2} + \left| b - \left(P - a - \frac{C_{\min} + C_{\max}}{2} \right) \right|$$

となる。イメージがつかない読者はグラフを参照されたい。



したがって、Bob は $P - a - \frac{C_{\min} + C_{\max}}{2}$ との差が最小となるようなコーンを選択すれば良い。これは事前に B をソートしておくことで二分探索で求められる。ソートにかかる計算量が $O(Y \log Y)$ 、各フレーバーについて二分探索にかかる計算量が $O(\log Y)$ であるから、全体の時間計算量は $O((X + Y) \log Y + Z)$ である。この解法を実装したのが C++ 実装例となる。

なお、 $\left| b - \left(P - a - \frac{C_{\min} + C_{\max}}{2} \right) \right|$ は b について単調減少関数と単調増加関数が連結した形であるから、最適な b の値の見当がつかない場合は三分探索を用いる事もできる。ソート後の B の index に対する三分探索を行う場合は、 B の重複する要素を削除する必要がある事に注意されたい。

また、別解としてあるスコアが実現可能かどうかを二分探索する解法が挙げられる。フレーバーを固定した時に、あるスコアを達成するコーンが存在するかは二分探索で判定できるので、時間計算量は

$$O(X \log Y \log(A_{\max} + B_{\max} + C_{\max}) + Y \log Y + Z)$$

である。この解法を実装したのが Python 実装例となる。



4

親密なシェフ (Intimate Chef)

Author : 蜂矢倫久 (Mitsubachi)

小課題 1

すべての 2 人組について、仲が悪いかどうかを判定し、仲が悪くなければその 2 人組を選んだときの満足度を求める。

仲が悪いかの判定は `vector` や `map` を用いれば $O(1)$ や $O(\log M)$ など可能である。また、ある 2 人組の満足度を求めるのは $O(1)$ で可能である。したがって、この部分は $O(N^2)$ や $O(N^2 \log M)$ などで行うことができる。

これによって求めることのできた満足度としてありうる候補の列を `sort` すればクエリあたり $O(1)$ で答えられる。全体の計算量は $O(N^2 \log N + Q)$ や $O(N^2(\log N + \log M) + Q)$ などになる。

小課題 2

説明を簡単にするために、 A_i は全て異なる値とする。また、 $A_1 \leq A_2 \leq \dots \leq A_N$ とする。このとき、満足度の分布は以下になる。

- ・ 満足度が $A_N + 1$ となる 2 人組はシェフ q がシェフ N となる $N - 1$ 組である。
- ・ 満足度が $A_{N-1} + 1$ となる 2 人組はシェフ q がシェフ $N - 1$ となる $N - 2$ 組である。
- ・ ...
- ・ 満足度が $A_2 + 1$ となる 2 人組はシェフ q がシェフ 2 となる 1 組である。

したがって、0 で初期化した変数に $N - 1$, $N - 2$ と足していき、 X_1 以上となったタイミングで、最後に足した値に対応する満足度を出力すれば良い。 A_i に同じ値がある場合でも同様に行うことができる。

A を `sort` する部分がボトルネックとなる。全体で $O(N \log N)$ で答えが求まる。

小課題 3

依頼することのできない M 組も含めた場合の満足度の分布は小課題 2 で述べたものである。すなわち、依頼することのできない M 組の部分そこから引くことで、満足度の分布を求めることができる。

満足度の分布を求めた後は小課題 2 と同様にして求めることができる。全体で $O(N \log N + M)$ で答えが求まる。



小課題 4

満足度の分布を求めた後を考える。これは非負整数からなる数列 $(v_1, v_2, v_3, \dots, v_n)$ と $w_1, w_2, \dots, w_m$ が与えられたとき、 $j = 1, 2, \dots, m$ について $v_1 + v_2 + \dots + v_i \geq w_j$ となる最小の i を求める問題と考えることができる。これは累積和を計算すれば二分探索を用いて各 j について答えを $O(\log n)$ で求めることができる。したがって、今回の問題においては全体で $O(N \log N + M + Q \log N)$ で答えが求まる。

小課題 5

満足度の最大値が分かれば良い。以降の小課題では、 $A_1 \leq A_2 \leq \dots \leq A_N$ となるようにシェフの番号を付け直すとする。

$q = j$ となる 2 人組のうち満足度が最大となる 2 人組を求めることを考える。シルパンチョの美味しさは A_j となるので、シェフ $1, 2, \dots, i-1$ について B_i が大きい順に調べていって、仲が悪い 2 人組でなければその時点でシェフ j に関する探索を打ち切れば良い。仲が悪い 2 人組かの判定は `map` を用いれば $O(\log M)$ で行える。

$p = i, q = j$ として調べた際に仲が悪い 2 人組であるためにその時点で探索を打ち切らないような (i, j) は高々 M 組である。したがって、この探索は $q = 1, 2, \dots, N$ 全体について $O((N + M) \log M)$ で行える。したがって、全体で $O((N + M) \log M)$ で答えが求まる。

小課題 6

小課題 5 と同様に $A_1 \leq A_2 \leq \dots \leq A_N$ となるようにシェフの番号を付け直すとする。

答えを二分探索することを考える。満足度が S 以上となる 2 人組が X_1 組以上あるかを判定できれば良い。

$q = j$ となる 2 人組のうち満足度が S 以上となる 2 人組の個数を求めることを考える。シルパンチョの美味しさは A_j となるので、シェフ $1, 2, \dots, i-1$ について B_i が大きい順に調べていって、仲が悪い 2 人組でないかつ満足度が S 未満となればその時点でシェフ j に関する探索を打ち切れば良い。仲が悪い 2 人組かの判定は `map` を用いれば $O(\log M)$ で行える。

探索の途中でも、満足度が S 以上となる 2 人組が X_1 組見つければ、その時点で全体の探索を打ち切って良い。したがって、二分探索の各ループは $O((N + M + X_1) \log M)$ で行える。

したがって、全体で

$$O((N + M + X_1) \log M \log(\max(A_i) + \max(B_i)))$$

で答えが求まる。



第 24 回日本情報オリンピック (JOI 2024/2025) 二次予選
2024 年 12 月 8 日 (オンライン開催)

小課題 7

小課題 5 と同様に $A_1 \leq A_2 \leq \dots \leq A_N$ となるようにシェフの番号を付け直すとする.

$X = \max(X_k)$ とする. 満足度が $1, 2, \dots, X$ 番目に高い 2 人組における満足度が分かれば良い. 小課題 6 の解法を用いることで, 満足度が S 以上となる 2 人組が X 組未満であるような最小の S を求めることができる. この S について, 満足度が S 以上となる 2 人組の個数を C とする.

このとき, 満足度が $C+1, C+2, \dots, X$ 番目に高い 2 人組における満足度は $S-1$ である. 満足度が $1, 2, \dots, C$ 番目に高い 2 人組における満足度を求めることを考える.

これは小課題 6 における二分探索の部分において, 条件を満たす 2 人組を見つけるたびにその 2 人組を **vector** などて記録すれば良い.

全体で $O((N+M+X) \log M \log(\max(A_i) + \max(B_i)) + Q)$ で答えが求まる. 実装によるが, 定数倍が良ければこの解法で満点を得られることも考えられる.

小課題 8

$X = \max(X_k)$ とする. 満足度が $1, 2, \dots, X$ 番目に高い 2 人組における満足度が分かれば良い. 仲の悪い 2 人組も認めた場合の $1, 2, \dots, M+X$ 番目に高い 2 人組における満足度が分かればそこから仲の悪い 2 人組の分を取り除けば良い. したがって, これを求めることを考える.

満足度が高いものを先頭におく **priority queue** などを用いて, 満足度が 1 番目に高い 2 人組から順に列挙することを考える. 前準備として, A_i と B_i に順序をつける. 同じ値については適当にタイブレークを行えば良い. ここで, $p = q$ となる 2 人組についても **priority queue** に追加することを許容する.

満足度が 1 番目に高い 2 人組について, A_i も B_i も 1 番目に高いペアである. したがって, これを最初 **priority queue** に追加すれば良い.

その後, $M+X$ 番目に高い満足度が分かるまで以下のアルゴリズムを繰り返せば良い.

1. **priority queue** の先頭の 2 人組を取り出す. シェフ p とシェフ q とする.
2. この 2 人組について既に調べたならば 1. へ移動する.
3. $p \neq q$ かつ仲の悪い 2 人組でない場合, これを残っている 2 人組の中で満足度が 1 番目に高い 2 人組とする.
4. A_p, A_q の A_i における順位を r_p, r_q として, B_p, B_q の B_i における順位を r'_p, r'_q とする.
5. $r_p \leq N-1$ ならば r_p+1 番目に A_i が高い i を p'_A とする. p'_B, q'_A, q'_B についても同様に定める.
6. シェフ p'_A, q の 2 人組, シェフ p'_B, q の 2 人組, シェフ p, q'_A の 2 人組, シェフ p, q'_B の 2 人組を **priority queue** に追加する.

全体で $O((N+M+X) \log(N+M+X))$ で答えが求まる.



5

衝突 (Collision)

Author : 蜂矢倫久 (Mitsubachi)

小課題 1

この小課題においては参加者の 1 ビョウあたりの速度は 1 か 2 であり、マラソン大会の時間は 1 ビョウである。したがって、重なりはゼッケン x を着用した 1 ビョウあたりの速度が 2 の人と、ゼッケン $x+1$ を着用した 1 ビョウあたりの速度が 1 の人が時刻 1 で同じ地点 $x+2$ にいる場合に限られる。

よって、人を追加したり削除する際の衝突の回数の変化は、その人がゼッケン A を着用しているとして、ゼッケン $A-1$ とゼッケン $A+1$ を着用している人がいるか、いる場合はその人の速度はいくつか分かれれば求められる。 $A=0$ の場合はゼッケン $L-1$ に関する情報、 $A=L-1$ の場合はゼッケン 0 に関する情報を参照するべきであることに注意せよ。

以上の操作は `map` などで可能である。 $O(N)$ や $O(N \log N)$ などで答えが求まる。

小課題 2

ゼッケン A を着用した 1 ビョウあたりの速度が S の人 P と、ゼッケン A' を着用した 1 ビョウあたりの速度が S' の人 P' との間で起こる衝突の回数を考える。

まず、 $S \leq S'$ として一般性を失わないので、 $S \leq S'$ が成り立っていると仮定する。このとき、 P' が P を 1 ビョウあたり $S' - S$ の速度で追いかけていると考えられる。すなわち、 $A' - A + t(S' - S)$ が L の整数倍となるような時刻 t に 2 人は衝突する。

このような t の個数は

$$\left\lfloor \frac{A' - A + T(S' - S)}{L} \right\rfloor - \left\lfloor \frac{A' - A}{L} \right\rfloor$$

となる。これは $O(1)$ で計算できる。

したがって、これを全ての 2 人組について計算することで、全体で $O(N^2)$ で答えが求まる。

小課題 3

小課題 2 の解法を変更のたびに行うと $O((N+Q)^2Q)$ となる。これを高速化することを考える。差分更新という解法を用いると高速化することが可能である。

人を追加するとき、2 人組の候補として増えるのはその人を含む $O(N+Q)$ 通りである。また、削除するときも 2 人組の候補として減るのはその人を含む $O(N+Q)$ 通りである。



第 24 回日本情報オリンピック (JOI 2024/2025) 二次予選

2024 年 12 月 8 日 (オンライン開催)

したがって、人の追加により増える、または人の削除により減る衝突の回数のみを求めれば、全体で $O((N+Q)^2)$ で答えが求まる。

小課題 4

小課題 2 の解法を高速化することを考える。参加者が 1 から n までの番号が付けられた n 人として、参加者 i がゼッケン A_i を着用し、1 ビョウあたりの速度は S_i とする。ここで $S_1 \leq S_2 \leq \dots \leq S_n$ が成立しているとして一般性を失わない。

このとき、人 i と人 j ($1 \leq i < j \leq n$) の間で起こる衝突の回数は

$$\left\lfloor \frac{A_j - A_i + T(S_j - S_i)}{L} \right\rfloor - \left\lfloor \frac{A_j - A_i}{L} \right\rfloor$$

と計算できる。これの総和を高速に計算したい。便宜的に衝突の回数を表す式を

$$\left\lfloor \frac{L + A_j - A_i + T(S_j - S_i)}{L} \right\rfloor - \left\lfloor \frac{L + A_j - A_i}{L} \right\rfloor$$

とする。

まず、 $\left\lfloor \frac{L + A_j - A_i}{L} \right\rfloor$ の総和を求めることを考える。 $0 \leq A_i, A_j \leq L-1$ より $\left\lfloor \frac{L + A_j - A_i}{L} \right\rfloor$ は 0 か 1 である。また、1 となる条件は $A_i \leq A_j$ であることである。したがって、転倒数を求めるのと同様のアルゴリズムで $O(N \log N)$ で計算できる。

次に、 $\left\lfloor \frac{L + A_j - A_i + T(S_j - S_i)}{L} \right\rfloor$ の総和を求めることを考える。 $A_i + S_i T = D_i L + E_i$, $0 \leq E_i \leq L-1$ となるように D_i, E_i を定める。これは $O(1)$ で計算できる。このとき、

$$L + A_j - A_i + T(S_j - S_i) = L(D_j - D_i) + L + E_j - E_i$$

となる。

したがって、

$$\left\lfloor \frac{L + A_j - A_i + T(S_j - S_i)}{L} \right\rfloor = D_j - D_i + \left\lfloor \frac{L + E_j - E_i}{L} \right\rfloor$$

となる。 $D_j - D_i$ の総和について、これは $D_i(2i - n - 1)$ を i について足し合わせたものである。

また、 $\left\lfloor \frac{L + E_j - E_i}{L} \right\rfloor$ の総和について、 $0 \leq E_i, E_j \leq L-1$ より $\left\lfloor \frac{L + A_j - A_i}{L} \right\rfloor$ の場合と同様にして求められる。

以上より、全体で $O(N \log N)$ で答えが求まる。

小課題 5, 小課題 6

満点となるアルゴリズムをこれから説明するが、定数倍が悪い場合もしくは実行時間の遅い言語では小課題 5 までしか実行時間に間に合わない可能性がある。

小課題 4 の解法を人の変更に対応させることを考える。まず、 X や Y を先読みすることで参加者の速度としてありうる値の一覧を作成しておく。

小課題 4 における $D_j - D_i$ の総和について考える。速度が s の人を追加するとき、それに伴う変化を求めることを考える。この人における D を d とする。また、速度が s 以下の人の人数とその人における D の総和をそれぞれ g, h とする。また、速度が $s+1$ 以上の人の人数とその人における D の総和をそれぞれ g', h' とする。



第 24 回日本情報オリンピック (JOI 2024/2025) 二次予選

2024 年 12 月 8 日 (オンライン開催)

このとき、 $D_j - D_i$ の総和の変化は $h' - g'd + gd - h$ となる。ここまでの議論で用いた g, h の値を計算する、またはクエリの変更に対応できるようにするには、各速度ごとに人数と D の総和を管理する Segment Tree や Binary Indexed Tree を作成すればよい。参加者の速度としてありうる値の一覧を用いて座標圧縮しておくと便利である。

人を削除するときも同様に考えることができる。クエリあたり $O(\log(N + Q))$ の計算量で処理できる。

$\left\lfloor \frac{L + A_j - A_i}{L} \right\rfloor$ の総和についても同様のことを考える。ゼッケン a を着用し、速度が s の人を追加するとき、それに伴う変化を求めることを考える。

小課題 4 における考察より、速度が s 以下で、 $a - 1$ 以下のゼッケンを着用する人数や、速度が $s + 1$ 以上で、 $a + 1$ 以上のゼッケンを着用する人数を求めることができれば良い。したがって、以下のような問題に帰着することができる。

2 次元平面上における以下のクエリを q 回処理せよ。それぞれのクエリは以下のパターン 1 またはパターン 2 のいずれかである。また、クエリの情報は前もって与えられるものとする（このような問題をオフラインクエリと言う）。

- ・ パターン 1: 2 次元平面上の点 (x, y) における重みを z 増加させる。
- ・ パターン 2: 2 次元平面上の $x_l \leq x \leq x_r, y_l \leq y \leq y_r$ を満たす点 (x, y) における重みを足し合わせた値を求める。

この問題には 2 種類の解法が考えられる。事前にパターン 1 で登場する x の集合と y の集合を求め、それに伴い座標圧縮を行うものとする。また、 i 番目のクエリをクエリ i と呼ぶ。

解法 1

クエリ分割統治と呼ばれるテクニックである。

クエリ 1 からクエリ $\left\lfloor \frac{q}{2} \right\rfloor$ におけるパターン 1 のクエリ全体がクエリ $\left\lfloor \frac{q}{2} \right\rfloor + 1$ からクエリ q におけるパターン 2 のクエリ全体に与える寄与を求めることを考える。

これは平面走査により $O(q \log q)$ で処理することができる。後は、問題のクエリを前半に限定したものと後半に限定したものを解けば元の問題を解くことができる。

したがって、 q 個のクエリを処理するのに必要な計算量を $T(q)$ とすると、

$$T(q) = 2T\left(\frac{q}{2}\right) + O(q \log q)$$

となる。したがって、 $T(q) = O(q(\log q)^2)$ となるので、 $O(q(\log q)^2)$ で解くことができる。



第 24 回日本情報オリンピック (JOI 2024/2025) 二次予選
2024 年 12 月 8 日 (オンライン開催)

解法 2

平方分割と呼ばれるテクニックである。

座標圧縮に伴い、2 次元平面の x 座標と y 座標をそれぞれ $O(\sqrt{q})$ 個の区間に分割する。これにより、2 次元平面は $O(q)$ 個の部分長方形に分けることができる。また、クエリをクエリ 1 からクエリ B まで、クエリ $B+1$ からクエリ $2B$ まで、... と $O\left(\frac{q}{B}\right)$ 個の区間に分ける。各区間のクエリを処理し終わるたびに、部分長方形における重みの総和に関して累積和を取る。

パターン 2 のクエリを処理することを考える。自分より前の区間にあり、かつパターン 2 のクエリにおける長方形に包含されている部分長方形における重みの総和は累積和により求められる。それ以外のパターン 1 のクエリにおける寄与は愚直に計算することができる。

したがって、 $O\left(\frac{q^2}{B} + qB\right)$ で解くことができる。 $B = \Theta(\sqrt{q})$ とすることで、 $O(q\sqrt{q})$ で解くことができる。

したがって、この問題は全体で

$$O((N+Q)(\log(N+Q))^2) \quad \text{または} \quad O((N+Q)\sqrt{N+Q})$$

で答えが求まる。実装の細かい部分については、解答例を参照せよ。

シラバス外の内容となるが、必要なところのみメモリを作る 2 次元動的 Segment Tree もしくは Binary Indexed Tree や、Wavelet Matrix などを用いて解く方法も考えられる。